Métodos Computacionais em Física de Altas Energias

Simulação de Eventos com Geradores

Marco Dias

7 de setembro de 2026

Introdução

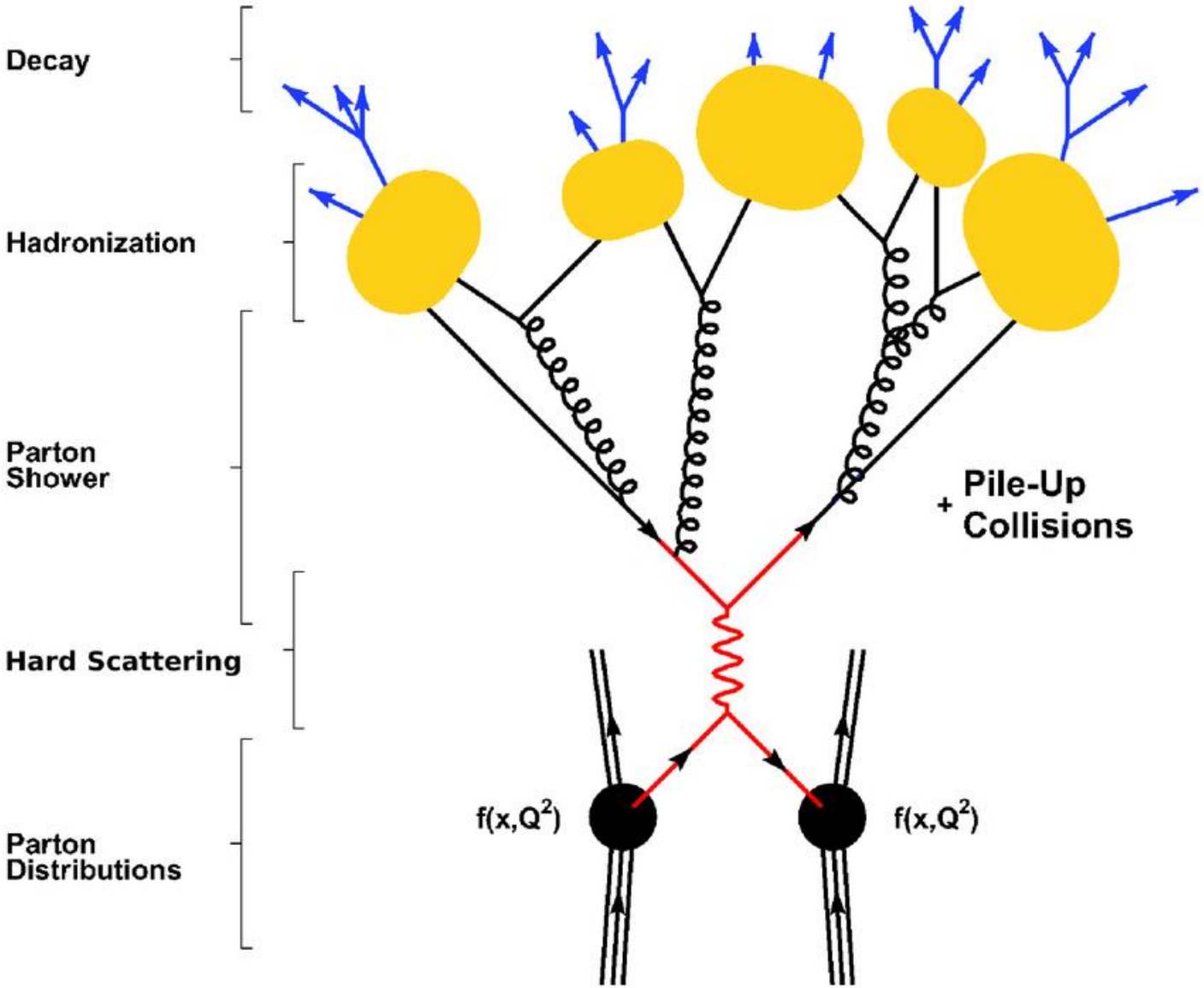
- Geradores de eventos simulam eventos de física de altas energias.
- A descrição perturbativa de árvore é simples, mas observações incluem:
  - Bremsstrahlung de fótons e glúons
  - Correções de diagramas de loop
- Fatoração permite calcular processos separadamente.
- Transições probabilísticas entre subprocessos são feitas com Monte Carlo.
- Partículas finais alimentam simulação do detector.

Subprocessos em Geradores Hadrônicos

Gerador Hadrônico Típico

- Composição e subestrutura do estado inicial
- Radiação do estado inicial
- O processo duro
- Decaimento de ressonâncias
- Radiação do estado final
- Processos semi-duros acompanhantes
- Hadronização e decaimentos posteriores

Subprocessos em Geradores Hadrônicos



Geradores Principais

- **Pythia6 e Pythia8**
- Herwig6, Herwig++, Herwig7
- Powheg
- Alpgen
- MG5_aMC@NLO
- CalcHEP

Aplicações dos geradores:

1. Dar uma ideia dos eventos esperados e suas taxas
2. Ajudar no planejamento de detectores
3. Ferramenta para estratégias de análise
4. Estimar correções de aceitação do detector
5. Interpretar fenômenos observados em termos de teoria fundamental

PYTHIA

- Focado em produção de múltiplas partículas em colisões elementares:
  - e^+e^-, pp, ep
- Gera eventos completos e detalhados
- Documentação em `https://pythia.org`
- Auto-contido, com interfaces para PDFs, decaimentos τ, etc.
- Filosofia: programa escravo, usuário fornece o programa principal
- Configuração via `Pythia::readString`

Uso do Pythia

1. **Inicialização**: `Pythia::init()`
   - Seleção de processos
   - Cinemática
   - PDFs
   - ligar ou desligar canais
2. **Laço de Eventos**: `Pythia::next()`
3. **Finalização**: saída e histogramas

Exemplo 1: Lista de Partículas

```
#include 'Pythia8/Pythia.h'
using namespace Pythia8;
int main() {
  Pythia pythia;
  pythia.particleData.list();
}
```

Exemplo 2: Produção de Z em PYTHIA

```
1 pythia.readString("Beams:eCM = 13000.");
2 pythia.readString("WeakSingleBoson:ffbar2gmZ = on");
3 pythia.readString("23:onMode = off");
4 pythia.readString("23:onlfMatch = -13 13");
5 pythia.readString("PhaseSpace:mHatMin = 80.");
6 pythia.readString("Next:numberShowEvent=0");
7 pythia.init();
```

Normalização de eventos

Fator de Peso

$$xsec = \text{Pythia.info.sigmaGen() [mb]}$$

$$CNOR = \frac{xsec \times 10^{12} \times LUMI}{NEV}$$

- *xsec*: seção de choque total integrada
- *NEV*: número de eventos
- *LUMI*: luminosidade escolhida

Exemplo 3: PDFs no PYTHIA

- Padrão: NNPDF2.3 QCD+QED LO
- Para mudar:

```
pythia.readString("PDF:pSet = value");
```

- Para PDF externo (LHAPDF):

```
pythia.readString("PDF:pSet = LHAPDF6:cteq6l1");
pythia.readString("SigmaProcess:alphaSvalue =
    0.1298");
```

Drell-Yan com PYTHIA

Processo Simulado

$$q\bar{q} \to \gamma^*/Z \to \mu^+\mu^-$$

- Selecionar eventos com $m_{\mu\mu} > 60$ GeV

1. Gerar eventos com Pythia8
2. Localizar os múons finais
3. Calcular componentes do quatro-momento
4. Calcular massa invariante do par
5. Salvar informações em uma `TTree`

Drell-Yan no Pythia8

```
#include "/home/mdias/Downloads/pythia8317/include/
    Pythia8/Pythia.h"
#include "TFile.h"
#include "TTree.h"
#include "TLorentzVector.h"

#include <cmath>
#include <iostream>

int main() {
    const int nEvents = 100000;

    Pythia8::Pythia pythia;

    pythia.readString("Beams:idA = 2212");
    pythia.readString("Beams:idB = 2212");
    pythia.readString("Beams:eCM = 13000.");
```

Drell-Yan no Pythia8

```
1   // Ativa a produção inclusiva de pares de múons.
2   pythia.readString("WeakSingleBoson:ffbar2gmZ = on");
3
4   // Permite o decaimento do bóson virtual
5   //ou Z em múons.
6   pythia.readString("23:onMode = off");
7   pythia.readString("23:onIfAny = 13");
8
9   // Apenas eventos com massa do sistema dileptônico
10  // acima de 60 GeV.
11  pythia.readString("PhaseSpace:mHatMin = 60.");
12
13  if (!pythia.init()) {
14      std::cerr << "Erro na inicializacao do Pythia8." << std::endl;
15      return 1;
16  }
```

Drell-Yan no Pythia8

```
1     TFile outputFile("pythia_drell_yan.root", "
          RECREATE");
2     TTree tree("events", "Eventos pp -> mu+ mu- + X");
3
4     int eventNumber = 0;
5     int nFinal = 0;
6
7     float muPlusPt = 0.0;
8     float muPlusEta = 0.0;
9     float muPlusPhi = 0.0;
10
11    float muMinusPt = 0.0;
12    float muMinusEta = 0.0;
13    float muMinusPhi = 0.0;
14
15    float invariantMass = 0.0;
16    float eventWeight = 1.0;
```

Drell-Yan no Pythia8

```
1    tree.Branch("eventNumber", &eventNumber, "
         eventNumber/I");
2    tree.Branch("nFinal", &nFinal, "nFinal/I");
3
4    tree.Branch("muPlusPt", &muPlusPt, "muPlusPt/F");
5    tree.Branch("muPlusEta", &muPlusEta, "muPlusEta/F"
         );
6    tree.Branch("muPlusPhi", &muPlusPhi, "muPlusPhi/F"
         );
7
8    tree.Branch("muMinusPt", &muMinusPt, "muMinusPt/F"
         );
9    tree.Branch("muMinusEta", &muMinusEta, "muMinusEta
         /F");
10   tree.Branch("muMinusPhi", &muMinusPhi, "muMinusPhi
         /F");
```

Drell-Yan no Pythia8

```
1   tree.Branch("invariantMass", &invariantMass, "
        invariantMass/F");
2   tree.Branch("eventWeight", &eventWeight, "
        eventWeight/F");
3
4   for (eventNumber = 0; eventNumber < nEvents; ++
        eventNumber) {
5       if (!pythia.next()) {
6           continue;
7       }
8
9       TLorentzVector muPlus;
10      TLorentzVector muMinus;
11
12      bool foundMuPlus = false;
13      bool foundMuMinus = false;
14
15      nFinal = 0;
```

Drell-Yan no Pythia8

```
        for (int i = 0; i < pythia.event.size(); ++i)
            {
    const Pythia8::Particle& particle = pythia.event[i
        ];

            if (!particle.isFinal()) {
                continue;
            }

            ++nFinal;
            const int pdgId = particle.id();

            TLorentzVector momentum;
            momentum.SetPxPyPzE(
                particle.px(),
                particle.py(),
                particle.pz(),
                particle.e()
            );
```

Drell-Yan no Pythia8

```
            if (pdgId == -13 && !foundMuPlus) {
                muPlus = momentum;
                foundMuPlus = true;
            }

            if (pdgId == 13 && !foundMuMinus) {
                muMinus = momentum;
                foundMuMinus = true;
            }
        }

        if (!foundMuPlus || !foundMuMinus) {
            continue;
        }

        muPlusPt = muPlus.Pt();
        muPlusEta = muPlus.Eta();
        muPlusPhi = muPlus.Phi();
```

Drell-Yan no Pythia8

```
            muMinusPt = muMinus.Pt();
            muMinusEta = muMinus.Eta();
            muMinusPhi = muMinus.Phi();

            invariantMass = (muPlus + muMinus).M();

            eventWeight = 1.0;
            tree.Fill();
        }

        outputFile.cd();
        tree.Write();
        outputFile.Close();

        pythia.stat();

        return 0;
}
```

MadGraph

- Gerador automático de diagramas de Feynman
- Calcula amplitudes no nível de elemento de matriz
- Usa `Pythia` para Parton showering e hadronização
- Gera arquivos no formato LHE (Les Houches Event)
- Permite incluir:
  - Canais de interação eletrofraca
  - Fóton virtual e bóson Z
- Download e geração de eventos
  `http://madgraph.phys.ucl.ac.be/`
- Inclusão em outras ferramentas de fenômenologia como
  `MadDM`,`MadAnalysis`, etc

Exercício com MadGraph

Processo Simulado

$$pp \to \mu^+\mu^-$$

- Incluindo canais com γ^* e Z

```
./bin/mg5_aMC
```

No prompt do MadGraph

```
import model sm
generate p p > mu+ mu-
output mg_drell_yan
launch
```

- MadGraph produz arquivo LHE com quatro-momentos do estado final

- MadGraph produz arquivo LHE com quatro-momentos do estado final

```
gunzip mg_drell_yan/Events/run_01/unweighted_events.lhe.gz
```

- Os resultados correspondem ao nível partônico, sem chuveiro partônico e sem hadronização.
- Em um programa em C++:
  1. Ler o arquivo LHE
  2. Identificar os múons
  3. Calcular:
     - p_T (momento transversal)
     - η (pseudorapidez)
     - ϕ (ângulo azimutal)
     - Massa invariante
  4. Armazenar resultados em uma TTree

Estrutura do arquivo LHE

- Cada evento contém um bloco semelhante a:

```
4  1  1.23456  91.1876  0.0073  0.118
```

- A primeira linha do bloco contém:
  1. número de partículas;
  2. identificador do processo;
  3. peso do evento;
  4. escala de fatorização;
  5. acoplamento eletromagnético;
  6. acoplamento forte.
- Cada linha de partícula contém, entre outras informações:
  1. identificador PDG;
  2. estado da partícula;
  3. índices das partículas-mãe;
  4. quadrimomento,
  5. energia E;
  6. massa.
- Os identificadores PDG relevantes são:$\mathrm{PDG}(\mu_-) = 13, \mathrm{PDG}(\mu_+) = -13$

Código para Leitura LHE

```
1  #include "TFile.h"
2  #include "TTree.h"
3  #include "TLorentzVector.h"
4
5  #include <cstdlib>
6  #include <fstream>
7  #include <iostream>
8  #include <sstream>
9  #include <string>
10
11 int main(int argc, char* argv[]) {
12     if (argc != 3) {
13         std::cerr
14             << "Uso: " << argv[0]
15             << " arquivo.lhe arquivo.root"
16             << std::endl;
17         return 1;
18     }
```

Código para Leitura LHE

```
1   const std::string lheFileName = argv[1];
2   const std::string rootFileName = argv[2];
3
4   std::ifstream lheFile(lheFileName);
5
6   if (!lheFile.is_open()) {
7       std::cerr << "Nao foi possivel abrir o arquivo
            LHE."
8                 << std::endl;
9       return 1;
10  }
11
12  TFile outputFile(rootFileName.c_str(), "RECREATE")
        ;
13  TTree tree("events", "Eventos gerados pelo
        MadGraph");
```

Código para Leitura LHE

```
1     int eventNumber = 0;
2     int nParticles = 0;
3
4     float muPlusPt = 0.0;
5     float muPlusEta = 0.0;
6     float muPlusPhi = 0.0;
7
8     float muMinusPt = 0.0;
9     float muMinusEta = 0.0;
10    float muMinusPhi = 0.0;
11
12    float invariantMass = 0.0;
13    float eventWeight = 1.0;
14
15    tree.Branch("eventNumber", &eventNumber, "
          eventNumber/I");
16    tree.Branch("nParticles", &nParticles, "nParticles
          /I");
```

Código para Leitura LHE

```
1     tree.Branch("muPlusPt", &muPlusPt, "muPlusPt/F");
2     tree.Branch("muPlusEta", &muPlusEta, "muPlusEta/F"
          );
3     tree.Branch("muPlusPhi", &muPlusPhi, "muPlusPhi/F"
          );
4
5     tree.Branch("muMinusPt", &muMinusPt, "muMinusPt/F"
          );
6     tree.Branch("muMinusEta", &muMinusEta, "muMinusEta
          /F");
7     tree.Branch("muMinusPhi", &muMinusPhi, "muMinusPhi
          /F");
8
9     tree.Branch("invariantMass", &invariantMass, "
          invariantMass/F");
10    tree.Branch("eventWeight", &eventWeight, "
          eventWeight/F");
```

Código para Leitura LHE

```
    std::string line;

    while (std::getline(lheFile, line)) {
        if (line.find("<event>") == std::string::npos)
            {
            continue;
        }

        if (!std::getline(lheFile, line)) {
            break;
        }

        std::stringstream header(line);

        int processId = 0;
        double factorizationScale = 0.0;
        double alphaQED = 0.0;
        double alphaQCD = 0.0;
```

Código para Leitura LHE

```
        header >> nParticles
               >> processId
               >> eventWeight
               >> factorizationScale
               >> alphaQED
               >> alphaQCD;

        TLorentzVector muPlus;
        TLorentzVector muMinus;

        bool foundMuPlus = false;
        bool foundMuMinus = false;

        for (int i = 0; i < nParticles; ++i) {
            if (!std::getline(lheFile, line)) {
                break;
            }
```

Código para Leitura LHE

```
1            std::stringstream particleLine(line);
2
3            int pdgId = 0;
4            int status = 0;
5            int mother1 = 0;
6            int mother2 = 0;
7            int color1 = 0;
8            int color2 = 0;
9
10           double px = 0.0;
11           double py = 0.0;
12           double pz = 0.0;
13           double energy = 0.0;
14           double mass = 0.0;
15           double lifetime = 0.0;
16           double spin = 0.0;
```

Código para Leitura LHE

```
1                particleLine
2                    >> pdgId
3                    >> status
4                    >> mother1
5                    >> mother2
6                    >> color1
7                    >> color2
8                    >> px
9                    >> py
10                   >> pz
11                   >> energy
12                   >> mass
13                   >> lifetime
14                   >> spin;
15
16               if (status != 1) {
17                   continue;
18               }
```

Código para Leitura LHE

```
            TLorentzVector momentum;
            momentum.SetPxPyPzE(px, py, pz, energy);

            if (pdgId == -13 && !foundMuPlus) {
                muPlus = momentum;
                foundMuPlus = true;
            }

            if (pdgId == 13 && !foundMuMinus) {
                muMinus = momentum;
                foundMuMinus = true;
            }
        }

        while (std::getline(lheFile, line)) {
            if (line.find("</event>") != std::string::
                npos) {
                break;
            }
```

Código para Leitura LHE

```
        if (!foundMuPlus || !foundMuMinus) {
            continue;
        }

        muPlusPt = muPlus.Pt();
        muPlusEta = muPlus.Eta();
        muPlusPhi = muPlus.Phi();

        muMinusPt = muMinus.Pt();
        muMinusEta = muMinus.Eta();
        muMinusPhi = muMinus.Phi();

        invariantMass = (muPlus + muMinus).M();

        tree.Fill();
        ++eventNumber;
    }
```

Código para Leitura LHE

```
    outputFile.cd();
    tree.Write();
    outputFile.Close();

    lheFile.close();

    std::cout << "Arquivo ROOT criado: "
              << rootFileName
              << std::endl;

    return 0;
}
```

Comparação PYTHIA vs MadGraph

Diferenças

- **PYTHIA**: inclui chuveiro partônico e hadronização
- **MadGraph**: apenas nível partônico (elemento de matriz)

Utilidade

- Comparar resultados com e sem efeitos de QCD
- Estudar correções de radiação

Resumo

- Geradores de eventos são essenciais na Física de Altas Energias
- PYTHIA oferece simulação completa (partônico + hadronização)
- MadGraph fornece precisão no elemento de matriz
- Exercícios práticos com:
  - Produção de Drell-Yan em PYTHIA
  - Análise de eventos LHE do MadGraph
  - Armazenamento em TTrees do ROOT
- Comparação entre níveis partônico e hadronizado